Combine Temporal Convolution and Self-attention for Action Segmentation Task

Anonymous CVPR 2021 submission

Paper ID 667

Abstract

Algorithms for the action segmentation task typically use temporal models to predict what action is occurring at each frame for a minute-long daily activity. The function of these temporal models can be decoupled into two aspects. The first is feature transformation, which aims to learn a mapping from low-level appearance feature space to the high-level semantic feature space. The second is feature aggregation that combines features from other timestamps to help embed the current frame. While recent researchers try to perform the action segmentation task based on temporal convolutions, we observe that the feature aggregation ability of temporal convolution is limited due to its inherent characteristics. To address the problem, we explore how to combine the self-attention operator with the great potential to make up for the shortcomings of temporal convolution on feature aggregation as a basic building module. Further, we propose a Multi-Stage Temporal Convolution Attention Network (MS-TCAN) to adapt the combined module efficiently for the action segmentation task. We conduct comprehensive experiments on three challenging datasets: 50Salads, GTEA and Breakfast, showing the superior performance of our approach.

1. Introduction

Algorithms for automatic detection and segmentation of human activities are crucial for applications that range from home security (e.g., to detect thefts, assaults, and suspicious activities), healthcare (e.g., to monitor the actions of elderly patients and detect life-threatening symptoms) to robot automatic (e.g., to analyze the behavior of humans and to interpret ongoing events). Different from action recognition that tries to classify action from a short trimmed video, the goal of the action segmentation task is to predict what action is occurring at each frame for a minute-long daily activity, as shown in Fig. 1.

Action segmentation methods typically use temporal models for frame labeling upon extracted frame-level features. In the action segmentation task, the function of these

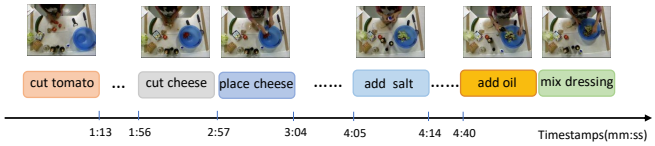

Figure 1. An example video for action segmentation task from the 50Salads [23] dataset. The goal is to predict what action is occurring at each frame.

temporal models can be decoupled into two aspects. The first is feature transformation, which aims to learn a mapping from low-level appearance feature space to the high-level semantic feature space. The second is feature aggregation that combines features from other timestamps to help embed the current frame. In recent years, researchers focus on variants of temporal convolutions as the basic building module for the action segmentation task, such as the encoder-decoder temporal convolution [15], deformable temporal convolution [17], or dilated temporal convolution [5, 11, 29]. Similar to modern Convolution Neural Network (CNN), researchers use a hierarchy of temporal convolutions to build Temporal Convolution Network (TCN) in their works. Those TCNs perform the feature transformation with multiple linear mappings and non-linear activation functions. And the hierarchy structure also allows it to capture long-range temporal dependencies for feature aggregation with the network depth increasing.

Although these TCNs show good results, they still have drawbacks. Looking deep into a single layer of temporal convolution, we observe three limitations in feature aggregation. *Firstly*, the convolution weights are fixed: the same filter kernel is applied across the whole video, which means that the feature aggregation for each frame cannot adapt to its content. The fixed convolution weights are not flexible enough for different frames with various contents. *Secondly*, the temporal convolution only operates on a small number of timestamps. Fig. 2(a) shows an example for a temporal convolution operation on the k_{th} frame with kernel size 3 and a dilation rate d. As we can see, the feature aggregation of the k_{th} frame can only adopt information

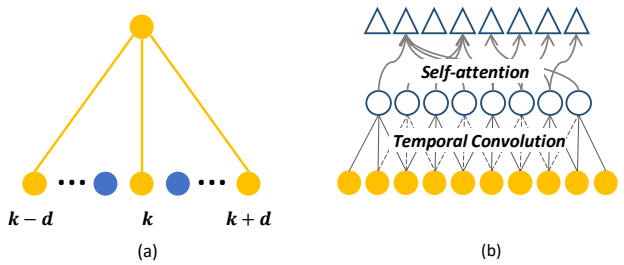

Figure 2. (a). The footprints for a temporal convolution with kernel size 3 and dilation rate d. (b). Self-attention serves as a remedial measurement for feature aggregation.

from two frames, *i.e.*, the $\{k-d\}_{th}$ and $\{k+d\}_{th}$ frame. The small number of timestamps may not be sufficient for the feature aggregation. *Thirdly*, when the temporal convolution is designed with a large dilation rate d to capture a larger receptive field (*i.e.*, $d = 512$ in [5, 29]), it is unavailable to aggregate information from its neighbors. With the above limitations on feature aggregation, some frames may not be well-embedded after the temporal convolution, making the temporal convolution a sub-optimum choice as the basic building module in the design of action segmentation models.

Recently, self-attention operator [26] has inspired related ideas for various computer vision tasks[32, 1, 7, 10, 31, 9, 28]. The key idea behind self-attention is to compute interaction weights with others and output a weighted average value. Unlike the convolution operator which performs feature aggregation by applying fixed weights to combine features from a set of nearby locations, self-attention generates weights dynamically according to the content of locations. This content-adapted way also allows self-attention to integrate features from all locations without increasing the number of parameters. Despite the potential of self-attention to make up for the shortcomings of temporal convolution on feature aggregation, it is not trivial to combine them as the basic building module and then further apply this module for action segmentation.

In this paper, we aim to combine temporal convolution and self-attention for the action segmentation task. The first problem is how to combine self-attention and temporal convolution as a basic module. To answer the question, we should have a clear understanding of their advantages and shortcomings. We observe that while self-attention is good at feature aggregation, its feature transformation ability is limited. Different from temporal convolution, the generated weights are shared across all channels, making self-attention essentially a linear combination function. Thus, the key idea of our combination is that temporal convolution focuses on feature transformation, while the self-attention operates on the transformed feature space to serve as the remedial measurement for feature aggregation, as shown in Fig. 2(b). Such a combination not only avoids the deficiency of self-attention on feature transformation, but also makes the full use of the ability of self-attention on feature aggregation from two aspects. First, it allows the self-attention to adaptively determine whether the frame is well-embedded after the temporal convolution. Then, it also offers a shortcut for the frame to learn a good embedding as it can directly integrate values from transformed feature space. The sequentially combined module makes the self-attention and temporal convolution complementary to each other.

Further, we propose a Multi-Stage Temporal Convolution Attention Network (MS-TCAN) to show how to adapt the combined module to modern TCNs. The MS-TCAN uses SOTA temporal convolution networks MS-TCN [5] as the backbone, and aims to improve the performance over a strong baseline. Similar to MS-TCN, MS-TCAN consists of one prediction stage and several refinement stages. Instead of simply replacing all temporal convolutions with the combined modules, many efforts have been made to make the combined module well suited for both the action segmentation task and the special multi-stage architecture. First, when applied to the action segmentation task, self-attention suffers from the under-fitting problem as well as the huge computation cost with the minute-long input video. To alleviate this, we propose a block operation before the self-attention, which splits the sequence into several clips and then performs self-attention separately. Motivated by the hierarchical design of TCNs, we gradually enlarge the length of clips with the network depth increasing. Second, the input of refinement stages and the prediction stage is different in the multi-stage architecture. The prediction stage operates on the video features to generate initial predictions while the inputs of remaining refinement stages are only the output probabilities of the previous stage. Thus, we use different designs to incorporate our combined module for the prediction stage and the refinement stage.

We conduct comprehensive experiments on three challenging datasets: GTEA [6], 50Salads [23] and Breakfast [12]. Experimental results demonstrate that our combination of self-attention and temporal convolution yields a notable performance gain against the strong baseline with acceptable extra FLOPs and GPU memories. Compared with other SOTA methods, MS-TCAN achieves comparable results on GTEA while achieving an encouraging improvement on 50Salads and Breakfast as the scale of datasets increases. In summary, our paper makes two main contributions:

1) To our best knowledge, this is the first attempt to combine self-attention and temporal convolution for action segmentation task.

2) We propose an effective MS-TCAN that adapts our design, and achieve SOTA results on three public benchmarks.

2. Related Works

Temporal Action Segmentation. Segmentation methods typically use temporal models for frame labeling upon extracted frame-level features. Earlier researches use Conditional Random Fields [25, 19, 21], HMM [13, 24], RNN [18, 27, 4, 22, 30] for action segmentation and classification. In recent years, motivated by the success of temporal convolution in speech synthesis, most of the works try to apply various temporal convolution networks for action segmentation task. For example, Lea *et al.* [15] first present a temporal convolution network for action segmentation by using an encoder-decoder architecture to capture long-range dependencies. Lei *et al.* [17] introduce a deformable convolution on the top of [15] and add a residual stream to the encoder-decoder. Farha *et al.* [5] propose a multi-stage architecture by stacking stages sequentially. They apply a series of dilated temporal convolutions in each stage, which enables the model to have a large temporal receptive field with fewer parameters. Wang *et al.* [29] reuse the structure of [5] as cascade stages. All these TCNs use temporal convolution as the basic building module for feature embedding. However, by decoupling the function of feature aggregation and feature transformation, we observe that temporal convolution has limitations in feature aggregation, making it a sub-optimum choice in the space of action segmentation models.

Self-attention. Self-attention operation originates from natural language processing [26]. Recently, self-attention has been applied to numerous computer vision areas, such as image recognition [32, 1], image segmentation [7, 10, 31, 9], and video prediction [28]. The applications of self-attention in image segmentation are the most related to ours. In these methods, forms of self-attention are primarily used as a global feature fusion strategy over the output of the last convolution layer. For example, Fu *et al.* [7] introduce a position attention module and a channel attention module to capture global dependencies in the spatial and channel dimensions, respectively. Yuan *et al.* [31] propose object context pooling that computes the weighted summation of the representations of all the pixels contained in the object context. In order to save the computation cost, Huang *et al.* [10] use a novel criss-cross attention module to harvest the contextual information of all the pixels on its criss-cross path, while Huang *et al.* [9] propose an interlaced sparse self-attention approach that factorizes the dense affinity matrix as the product of two sparse affinity matrices to improve the efficiency of the self-attention mechanism. Inspired by their works, we further explore the connections between self-attention and temporal convolution for the action segmentation task by decoupling the functions of feature aggregation and feature transformation.

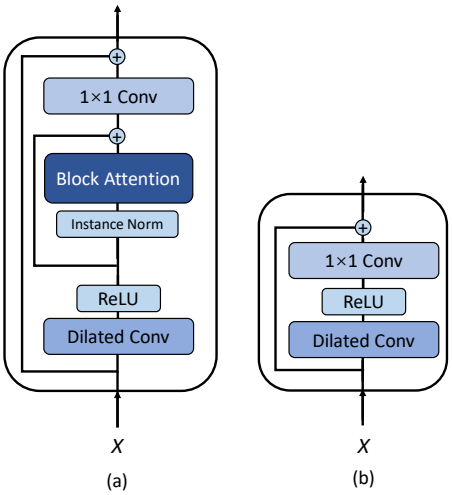

Figure 3. (a). Overview of the sequentially combined module of temporal convolution and self-attention. (b). Overview of the basic convolution-based module.

3. Method

In this section, we propose a Multi-Stage Temporal Convolution Attention Network (MS-TCAN). As we discussed in Sec. 1, MS-TCAN adopts the combination of temporal convolution and self-attention as the basic building module. To demonstrate the effectiveness, MS-TCAN uses the state-of-the-art MS-TCN [5] as the backbone and aims to improve the performance over a very strong baseline. Similar to MS-TCN, our proposed MS-TCAN consists of one prediction stage and several refinement stages. Firstly, we describe the prediction stage of MS-TCAN in Sec. 3.2. Then, we further show how to expand our idea with the rest refinement stages in Sec. 3.3.

3.1. Video Encoding

To save memory consumption, we first extract the low-level visual features with an off-the-shelf pre-trained video network. Following the previous works [15, 17, 5, 29], we use the I3D feature [2]. Then the extracted video features $X = \{x_1, ..., x_T\} \in \mathbb{R}^{D \times T}$ are fed into MS-TCAN for temporal action segmentation, where T is the video length and $D = 2048$ is the feature dimension.

3.2. Prediction Stage

We first introduce the combined module of temporal convolution and self-attention in the prediction stage, as shown in Fig.3(a). We denote it as *seq-conv-attention* module. Similar to the basic *convolution-based* module (Fig. 3(b)) in MS-TCN [5], each *seq-conv-attention* module first applies a dilated convolution with ReLU activation to the input X, and end with a 1x1 convolution layer with the resid-

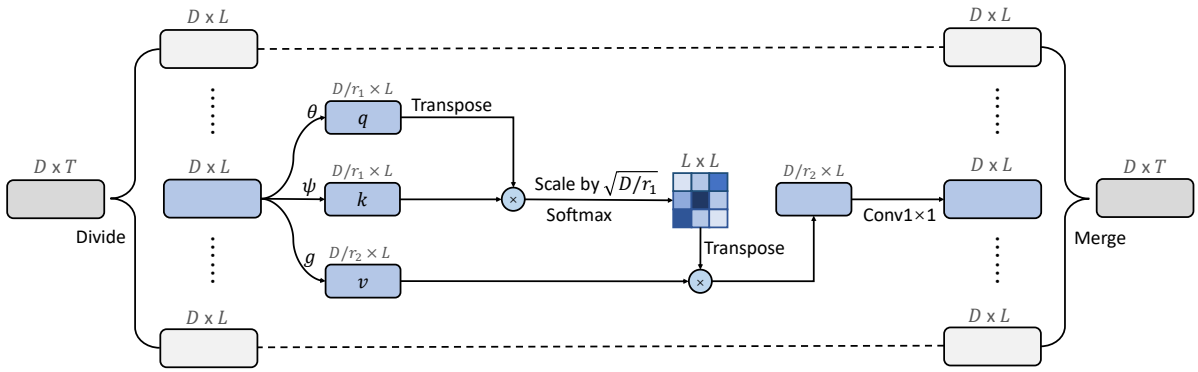

Figure 4. Overview of the block self-attention operator. We first divide the input sequence into N clips and the perform standard dot-product self-attention for each clip separately.

ual connection. For the dilated convolution in the *seq-conv-attention* module, we use the same design as the basic *convolution-based* module, which uses acausal convolutions with kernel size 3. As we discussed in Sec. 1, the self-attention is sequentially stacked after the temporal convolution with residual connections to facilitate gradients flow. By directly operating on the transformed output, it allows the full use of self-attention to make up for the shortcomings of temporal convolution while avoiding its deficiency in feature transformation. In particular, it enables the self-attention to adaptively determine whether the frame is well-embedded after the temporal convolution. Moreover, it offers a shortcut for the frame to learn a good embedding as it can directly integrate values from transformed feature space. The set of operations at the conv-attention module can be formally described as follows:

$$\begin{aligned} \text{out} &= \text{ReLU}(\text{dilated_conv}(X)), \\ \hat{\mathbf{X}} &= \mathbf{norm}(\mathbf{out}), \\ \mathbf{out} &= \mathbf{block_att}(\hat{\mathbf{X}}) + \mathbf{out}, \\ \text{out} &= X + \text{conv1} \times 1(\text{out}). \end{aligned} \tag{1}$$

The self-attention is incorporated with a block operation in each *seq-conv-attention* module. The computation complexity of the self-attention mechanism is about $O(T^2)$ given input of length T, and the cost can become very huge for the video sequence that lasts for minute-long. Also, when modeling the extremely long sequence, the self-attention will suffer from a severe under-fitting problem as shown in our experiment. The block operation divides the input sequence $\hat{X}$ of N clips $\hat{X}_1, ..., \hat{X}_N$ with equal length L and then performs self-attention operator on each clip separately, as shown in Fig. 4. With the block self-attention, the computation cost drops to $O(TL)$ for each *seq-conv-attention* module. We use the standard dot-product self-attention in our *seq-conv-attention* module. Specifically, for each clip $\hat{X}_i \in \mathbb{R}^{D \times L}$, we apply three projections to obtain key ($k \in \mathbb{R}^{D/r_1 \times L}$), query ($q \in \mathbb{R}^{D/r_1 \times L}$), and value ($v \in \mathbb{R}^{D/r_2 \times L}$) representations, where r_1 and r_2 are the hyper-parameters to decide the dimension of three representations. We set $r_1 = 2$ and $r_2 = 1$ for all experiments. Next, we calculate the dot products of the query with all keys, divide each by $\sqrt{D/r_1}$ to stabilize training, and apply a softmax function to obtain the weights of every two frames. Finally, we compute a weighted sum using the output of the value projection v for each frame:

$$\text{Attention}(q, k, v) = \text{softmax}(\frac{k^T q}{\sqrt{D/r_1}})v \tag{2}$$

Now, we describe the pipeline of the prediction stage with the *seq-conv-attention* module. The first layer of the prediction stage is a 1x1 temporal convolution layer, which adjusts the dimension of the input feature. This layer is then followed by a series of *seq-conv-attention* modules to carry out the feature embedding process. The dilation factor d of the dilated convolution is doubled at each *seq-conv-attention* module, *i.e.*, $d = 1, 2, ..., 512$. Similarly, the length L of each clip for the block operation is also doubled at each *seq-conv-attention module* with the network depth increasing, *i.e.*, $L = 1, 2, 4, ..., 512$. Such a design enables the feature aggregation process to focus on local neighbors at first, and then gradually enlarge its footprints to capture longer dependencies. Finally, to get the probabilities $Y \in \mathbb{R}^{C \times T}$ for the output class, where C is the number of classes, we apply a 1×1 convolution over the output of the last *seq-conv-attention* module followed by a softmax activation.

3.3. Refinement Stages

The idea of multi-stage architecture is composing several refinement models sequentially, and each refinement model operates directly on the output probabilities of the previous one to refine the predictions incrementally. In the implementation of MS-TCN [5], refinement stages share the

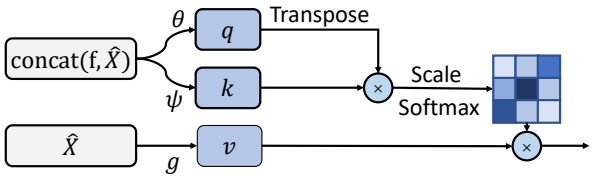

Figure 5. Self-attention operator in the *seq-conv-attention-v2* module. f is the high-level features of the first stage. $\hat{X}$ is the input of the self-attention.

same network architecture with the prediction stage, which is denoted as a single-stage TCN (SS-TCN). However, we cannot directly apply the *seq-conv-attention* module to refinement stages. Because the input of the refinement stages is only the probabilities, which is far from enough to learn useful representations q, k, v to perform feature aggregation. Thus, we design a variant *seq-conv-attention* module for the refinement stages, denoted as *seq-conv-attention-v2* module for better explanation.

Typically, the *seq-conv-attention-v2* module has almost the same architecture with the *seq-conv-attention* module. The only difference is in the self-attention operator, as shown in Fig. 5. f is the high-level feature output by the last conv-attention module in the first stage. $\hat{X}$ is the output of the temporal convolution. We use the concatenation of f and $\hat{X}$ to get the representations q and k, while the representation v is only obtained from $\hat{X}$. In this way, the high-level feature f serves as the guidance of the refinement. We will show the effects of passing high-level features.

In particular, the *seq-conv-attention-v2* module is only applied to the first refinement stage. The input probabilities of the first refinement stage have the strong correspondence with the high-level feature f, so that f can be used to guide the refinement of the probabilities by participating in the calculation of q, k representation. However, for the rest of refinement stages, such correspondence may not exist anymore after the probabilities are further refined in the first refinement stage. Therefore, we use the basic SS-TCN in [5] without the self-attention operator for rest refinement stages. We will show the performance of different designs of multi-stage architecture in our experiments.

3.4. Loss Function and Channel Masking Layer

As a loss function, we use the same loss function in MS-TCN [5], which consists of a basic cross-entropy loss and a smoothing loss for reducing the over-segmentation error. Following the best configurations in MS-TCN, our Multi-stage TCAN consists of 4 stages (one prediction stage and 3 refinement stages) and 10 modules per stage. We additionally apply a special dropout layer before the MS-TCAN, denoted as the channel masking layer. Different from regular dropout, the channel masking layer randomly masks the entire semantic channel for the input feature. We observe that channel masking is efficient for high-dimension input features, *i.e.*, $2048d$ for I3D features, as it can help decouple the semantic information among different channels and enhance the independence of each channel.

4. Experiments

Datasets. We conduct the experiment on three public datasets: 50Salads [23], Georgia Tech Egocentric Activities (GTEA) [6], and the Breakfast dataset [12].

50Salads dataset contains 50 videos with 17 action classes of users making a salad and has been used for both action segmentation and detection. On average, each video contains 20 action instances and is about 6.4 minutes long. These activities were performed by 25 actors where each actor prepared two different salads. For evaluation, we perform 5-fold cross-validation and report the average results.

GTEA dataset contains 28 videos of 11 action classes of daily activities in a kitchen, like take or pour, performed by 4 subjects. On average, each video has 20 action instances. GTEA is the smallest dataset among three datasets. The total number of frames in the dataset is 31,222 and each video is about half-minute long on average. We use the standard 4 different train-test splits as previous works by leaving one subject out. Similar to 50Salads, the average results of 4 splits are reported.

Breakfast dataset is the largest and the most challenging dataset among the three datasets with 1712 videos. The videos were recorded in 18 different kitchens, showing breakfast preparation related activities. Overall, there are 48 different actions, and each video contains 6 action instances on average. We use the standard 4-fold cross-validation for evaluation.

Evaluation Metrics. For all the three datasets, the following three evaluation metrics are used as in previous works [15]: frame-wise accuracy, segmental edit score, and segmental overlap F1 score with threshold k/100, denoted as F1@k. The frame-wise accuracy is the most common metric, but is not sensitive to the over-segmentation errors. The F1@k metric penalizes the over-segmentation errors, however, it ignores minor temporal shifts between the predictions and ground truth. The segmental edit score metric overcomes above limitations, and can be used to measure the quality of the predictions.

Training Details. Our MS-TCAN is implemented with PyTorch and can be trained in an end-to-end manner. We use Adam optimizer and batch size 1 for all three datasets. The initial learning rate is set to 5e-4 and multiplied by 0.5 when the loss stops decreasing in 3 epochs. We train the MS-TCAN with 150 epochs for 50Salads and Breakfast, while 90 epochs for GTEA. The dropout rate of channel masking is set to 0.3 for 50Salads and Breakfast, 0.5 for GTEA.

Baseline. Noted that we additionally apply a channel masking layer with our MS-TCAN. To justify our method's ability adequately, we use the MS-TCN[5] with the channel masking layer as our baseline, denoted as MS-TCN*cm* for following comparison.

4.1. Study on the Combination of Self-attention and Temporal Convolution

To better understand the connections between self-attention and temporal convolution, we propose a *para-conv-attention* module as shown in Fig. 6. In the *para-conv-attention* module, the self-attention and temporal convolution are combined in a parallel manner, where the self-attention directly operates on the raw input X and then adds up to the transformed output of the temporal convolution. The design of *para-conv-attention* module does not follow our key idea, because it requires the self-attention to perform feature transformation and feature aggregation simultaneously and cannot cooperate with the temporal convolution due to the parallel flow.

We conduct experiments on 50Salads by replacing the *seq-conv-attention* module with *para-conv-attention* in MS-TCAN. In order to give an intuitive comparison, we also present the results of the baseline model MS-TCN*cm*. Results are shown in Table 1. The MS-TCAN that adapts *seq-conv-attention* yields the best performance with a large margin compared to the baseline MS-TCN*cm* (up to 4.7% in Edit Score and 2.7% in Frame-wise accuracy). However, the MS-TCAN that adapts *para-conv-attention* modules only achieves comparable performance with the baseline. This highlights the impact of the proposed *seq-conv-attention* module in improving the quality of the predictions. Experimental results also demonstrate that the improvement of combining self-attention is not simply due to the increase of parameters. Our proposed combination of self-attention and temporal convolution achieves superior results because we have a clear understanding of their advantages and shortcomings. By decoupling the function of feature transformation and feature aggregation, we believe that one effective way to combine self-attention is to serve it as the remedial measurement for feature aggregation after the temporal convolution.

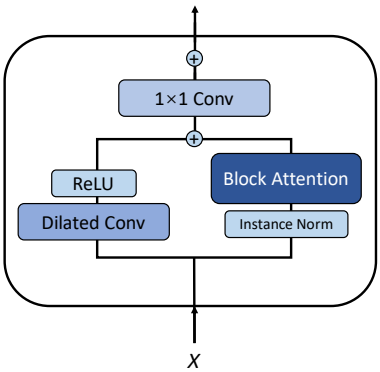

Figure 6. Overview of the *para-conv-attention* module where the self-attention and temporal convolution is combined is a parallel manner.

50Salads	F1@{10, 25, 50}	Edit Score	Acc.
MS-TCN*cm*	80.4 78.4 71.4	74.4	83.5
para-conv-attention	81.4 79.4 71.4	72.4	84.3
seq-conv-attention	**85.3 83.4 76.4**	**79.1**	**86.2**

Table 1. Comparison of *seq-conv-attention* module and *para-conv-attention* module on the 50Salads dataset.

50Salads	F1@{10, 25, 50}	Edit Score	Acc.	Mem.
No-block	80.8 78.8 72.0	74.6	86.0	30G
Block	**85.3 83.4 76.4**	**79.1**	**86.2**	2.5G

Table 2. Impact of block operation on the 50Salads dataset.

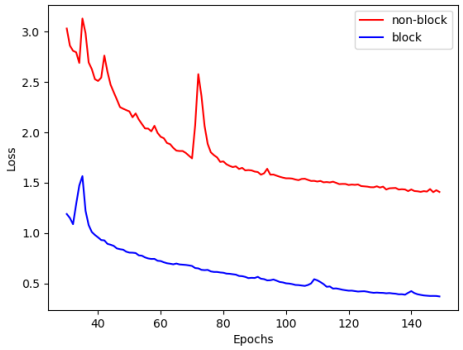

Figure 7. Training losses on 50Salads dataset for whether using block operation. Best viewed in color.

4.2. Effects of Block Self-attention

In our implementation of the *seq-conv-attention* module, the block self-attention first divides the input sequence $\hat{X} \in \mathbb{R}^{D\times T}$ into N clips $\hat{X}_1, ..., \hat{X}_N$ with equal length L and then performs self-attention operator for each clip separately. In this section, we try to apply self-attention on the full-length input without the block operation in each *seq-conv-attention* module. Results on 50Salads are shown in Table 2. Despite the huge computational costs (need about 30G GPU memory to train on 50Salads with batch size 1, which is 10 times larger than using the block operation), the performance also drops a lot. We further observe that the performance drop is mainly caused by the under-fitting problem. We plot the training loss of whether using block operation under the same training settings, as shown in Fig. 7. The training losses without block operation quickly converged to a high value and is higher than training losses that applies block operation. The qualitative

loss curve shows that self-attention may come into question when it is directly applied to the action segmentation task with minute-long videos.

4.3. Designs for Multi-stage Architecture

In this section, we try to explore different designs for multi-stage architecture. As introduced in Sec. 3.3, when expanding our idea to multi-stage architecture, we use a variant *seq-conv-attention-v2* module to pass high-level features to the first refinement stage while keeping the rest refinement stages basic SS-TCN. To demonstrate the effectiveness, we specify the following variants for comparison, as shown in Fig. 8.

- MS-TCAN*v1*: Apply *seq-conv-attention* module in the prediction stage, and apply basic SS-TCN[5] for all refinement stages.
- MS-TCAN*v2*: Apply *seq-conv-attention* module for all stages.
- MS-TCAN*v3*: Apply *seq-conv-attention* module in the prediction stage, and apply *seq-conv-attention-v2* module for all refinement stages.
- MS-TCAN: Our model.

Results on 50Salads dataset are shown in Table 3. Compared with MS-TCAN*v1* which only uses basic SS-TCN for refinement stages, the performance of MS-TCAN*v2* and MS-TCAN*v3* drops a lot. The results are consistent with our analysis in Sec. 3.3. The special multi-stage structure brings great difficulties for us to incorporate self-attention into refinement stages. An inappropriate design may make

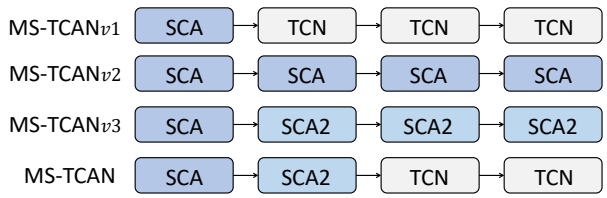

Figure 8. Different designs for multi-stage architecture. SCA denotes the *seq-conv-attention* module. SCA2 denotes the *seq-conv-attention-v2* module. TCN denotes the SS-TCN [5] that without self-attention operator.

50Salads	F1@{10, 25, 50}	Edit Score	Acc.
MS-TCAN*v1*	84.2 82.2 74.5	77.7	85.7
MS-TCAN*v2*	80.5 78.2 70.0	74.1	83.4
MS-TCAN*v3*	81.8 80.8 72.8	76.3	84.2
MS-TCAN	**85.3 83.4 76.4**	**79.1**	**86.2**

Table 3. Comparison of different designs for multi-stage architecture on 50Salads dataset.

50Salads	F1@{10, 25, 50}	Edit Score	Acc.
MS-TCN	76.3 74.0 64.5	67.9	80.7
MS-TCN*cm*	80.4 78.4 71.4	74.4	83.5
MS-TCAN	**85.3 83.4 76.4**	**79.1**	**86.2**

Table 4. Ablation study on 50Salads dataset.

the self-attention have the negative effects in refinement stages. Meanwhile, our final MS-TCAN yields the best performance by only applying *seq-conv-attention-v2* module to the first refinement stage, because it allows incorporating high-level features of the first stage to guide the refinement in the first refinement stage.

4.4. Ablation Study

In this section, we demonstrate the ability of our proposed MS-TCAN by comparing it to its variants. We specify the following baseline and variants: 1) MSTCN: MS-TCN [5] backbone with 4 stages and 10 modules per stage. 2) MS-TCN*cm*: MS-TCN with the channel masking layer. 3) MS-TCAN: Our model. Results on the 50Salads dataset are summarized in Table 4.

Comparison between MS-TCN and MS-TCN*cm*. Comparing MS-TCN with MS-TCN*cm*, the channel masking layer achieves surprising performance improvement. For the high dimension input feature, randomly masking some channels helps decouple the semantic information between different channels. The channel masking layer also can prevent the over-fitting problem as we further increase the parameters by incorporating self-attention.

Comparison between MS-TCN*cm* and MS-TCAN. MS-TCAN further yields notable performance gain against the MS-TCN*cm*. We observe a large improvement in F1 score and Edit Score, which indicates the effectiveness of our solution in reducing over-segmentation errors. This may benefit from the unlimited footprints of self-attention as it can integrate features from local neighbors.

4.5. Comparison on Single-Stage Model

In the previous section, we have seen that combining self-attention and temporal convolution achieves better results with the multi-stage architecture. In this section, we investigate the effects of our solution for the single-stage model. We compare the performance between a single-stage TCN with channel masking(denoted as SS-TCN*cm*) in MS-TCN and the prediction stage with channel masking of MS-TCAN(denoted as PS-TCAN*cm*). The results on the 50Salads dataset are shown in Table 5. The PS-TCAN*cm* outperforms SS-TCN*cm* by a more obvious margin without the rest refinement stages. This demonstrates the effectiveness of our motivation and our solution. This effect can also be reflected in the qualitative results in Fig. 9.

50Salads	F1@{10, 25, 50}	Edit Score	Acc.
SS-TCN*cm*	27.0 25.8 22.8	19.7	83.4
PS-TCAN*cm*	**51.4 50.0 45.2**	**41.4**	**84.9**

Table 5. Comparison for the single-stage model on 50Salads dataset.

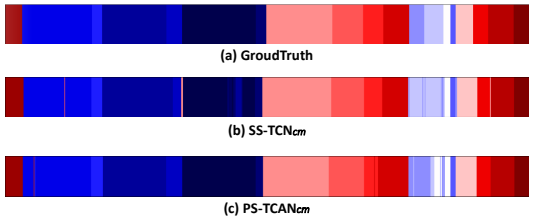

Figure 9. Qualitative result from the 50Salads dataset for comparing different single-stage models.

4.6. Comparison with SOTA

This section compares the proposed MS-TCAN to the state-of-the-art methods on three datasets: 50Salads, GTEA, and Breakfast datasets. The results are presented in Table 6. Our model outperforms the state-of-the-art methods on the three datasets. We further observe a large improvement in Edit Score. This is consistent with our analysis. The self-attention can integrate features from all locations especially from the local neighbors, which helps reduce over-segmentation errors. Compared to the improvement on 50Salads(up to 4.8% on Edit Score) and Breakfast(up 6.9% on Edit Score), MS-TCAN is only slightly better on GTEA dataset. This might be due to the small size of GTEA dataset which only contains 31,222 frames(518,411 frames for 50salads, more than 4,000,000 frames for breakfast). And the small size of the dataset might result in an over-fitting problem due to the increase of parameters. Breakfast is the most challenging dataset compared to the others and there remains a large room for improvement.

4.7. Size of the Model and Computation Cost

The benefits of combining self-attention far outweigh the computational burden it brings. Table 7 illustrates the computation cost of MS-TCAN and the backbone MS-TCN [5] during training on 50Salads dataset. We can observe that the self-attention in MS-TCAN brings about 40M parameters and 40M FLOPs, which is acceptable for modern computer chips. And thanks to the block operation, MS-TCAN only acquires additional 0.7G GPU memory on 50Salads dataset with batch size 1. Also, the self-attention can be effectively formulated as a matrix multiplication operation, which can be accelerated by the low-level frameworks.

50Salads	F1@{10,25,50}			Edit Score	Acc.
IDT+LM [20] *CVPR,2016*	44.4	38.9	27.8	45.8	48.7
ST-CNN [16] *ECCV,2016*	55.9	49.6	37.1	45.9	59.4
Bi-LSTM [22] *CVPR,2016*	62.6	58.3	47.0	55.6	55.7
ED-TCN [15] *CVPR,2017*	68.0	63.9	52.6	59.8	64.7
TDRN [17] *CVPR,2018*	72.9	68.5	57.2	66.0	68.1
SSA-GAN [8] *PR,2020*	74.9	71.7	67.0	69.8	73.3
MS-TCN [5] *CVPR,2019*	76.3	74.0	64.5	67.9	80.7
BCN [29] *ECCV,2020*	82.3	81.3	74.0	74.3	84.4
MS-TCAN	**85.3**	**83.4**	**76.4**	**79.1**	**86.2**
GTEA	F1@{10,25,50}			Edit Score	Acc.
ST-CNN [16] *ECCV,2016*	58.7	54.4	41.9	-	60.6
Bi-LSTM [22] *CVPR,2016*	66.5	59.0	43.6	-	55.5
ED-TCN [15] *CVPR,2017*	72.2	69.3	56.0	-	64.0
TDRN [17] *CVPR,2018*	79.2	74.4	62.7	74.1	70.1
SSA-GAN [8] *PR,2020*	80.6	79.1	74.2	76.0	74.4
MS-TCN [5] *CVPR,2019*	85.8	83.4	69.8	79.0	76.3
BCN [29] *ECCV,2020*	88.5	87.1	77.3	84.4	**79.8**
MS-TCAN	**89.7**	**88.5**	**77.8**	**85.6**	79.6
Breakfast	F1@{10,25,50}			Edit Score	Acc.
ED-TCN [15] *CVPR,2017*	-	-	-	-	43.3
HTK [14] *CVIU,2017*	-	-	-	-	50.7
TCFPN [3] *CVPR,2018*	-	-	-	-	52.0
HTK(64) [13]*WACV,2016*	-	-	-	-	56.3
GRU *CVPR,2017*	-	-	-	-	60.6
MS-TCN [5] *CVPR,2019*	52.6	48.1	37.9	61.7	66.3
BCN [29] *ECCV,2020*	68.7	65.5	**55.0**	66.2	70.4
MS-TCAN	**73.2**	**67.2**	53.7	**73.1**	**71.3**

Table 6. Comparison with the state-of-the-art on 50Salads, GTEA and the Breakfast dataset.

Methods	#params (x10^6)	FLOPs (x10^9)	Mem. (xG)
MS-TCAN	118	1.19	2.4
MS-TCN	81	0.81	1.7

Table 7. Comparison of MS-TCAN and MS-TCN on number of parameters, FLOPs, and GPU Memory cost during training on 50Salads dataset.

5. Conclusion

This paper addresses the action segmentation task, which is to predict what action is occurring at every frame for complex daily activity. By decoupling the functions of feature transformation and feature aggregation, we observe that the temporal convolution is limited in feature aggregation ability, while the self-attention technique holds the huge potential to make up those shortcomings. We propose an effective combination of temporal convolution and self-attention as the basic module for the feature embedding process, and adopt such a combination in a modern temporal convolution network for the action segmentation task. Experimental results show the effectiveness of our solution.

References

[1] Irwan Bello, Barret Zoph, Quoc Le, Ashish Vaswani, and Jonathon Shlens. Attention augmented convolutional networks. In *IEEE International Conference on Computer Vision (ICCV)*, pages 3285–3294, 2019. 2, 3

[2] João Carreira and Andrew Zisserman. Quo Vadis, action recognition? a new model and the kinetics dataset. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4724–4733, 2017. 3

[3] Li Ding, Chenliang Xu, and Juergen Gall. Weakly-supervised action segmentation with iterative soft boundary assignment. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, page 6508–6516, 2018. 8

[4] Jeff Donahue, Lisa Anne Hendricks, Marcus Rohrbach, Subhashini Venugopalan, Sergio Guadarrama, Kate Saenko, and Trevor Darrell. Long-term recurrent convolutional networks for visual recognition and description. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, 39(4):677–691, 2017. 3

[5] Yazan Abu Farha and Jurgen Gall. MS-TCN: multi-stage temporal convolutional network for action segmentation. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 3575–3584, 2019. 1, 2, 3, 4, 5, 6, 7, 8

[6] Alireza Fathi, Xiaofeng Ren, and James M. Rehg. Learning to recognize objects in egocentric activities. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 3281–3288, 2011. 2, 5

[7] Jun Fu, Jing Liu, Haijie Tian, Yong Li, Yongjun Bao, Zhiwei Fang, and Hanqing Lu. Dual attention network for scene segmentation. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 3141–3149, 2019. 2, 3

[8] Harshala Gammulle, Simon Denman, Sridha Sridharan, and Clinton Fookes. Fine-grained action segmentation using the semi-supervised action GAN. *Pattern Recognition*, 98:107039, 2020. 8

[9] Lang Huang, Yuhui Yuan, Jianyuan Guo, Chao Zhang, Xilin Chen, and Jingdong Wang. Interlaced sparse self-attention for semantic segmentation. *arXiv*, 2019. 2, 3

[10] Zilong Huang, Xinggang Wang, Yunchao Wei, Lichao Huang, Humphrey Shi, Wenyu Liu, and Thomas S. Huang. CCNet: Criss-cross attention for semantic segmentation. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, pages 1–1, 2020. 2, 3

[11] Noureldien Hussein, Efstratios Gavves, and Arnold W.M. Smeulders. Timeception for complex action recognition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 254–263, 2019. 1

[12] Hilde Kuehne, Ali Arslan, and Thomas Serre. The language of actions: Recovering the syntax and semantics of goal-directed human activities. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 780–787, 2014. 2, 5

[13] Hilde Kuehne, Juergen Gall, and Thomas Serre. An end-to-end generative framework for video segmentation and recognition. In *IEEE Winter Conference on Applications of Computer Vision (WACV)*, pages 1–8, 2016. 3, 8

[14] Hilde Kuehne, Alexander Richard, and Juergen Gall. Weakly supervised learning of actions from transcripts. *Computer Vision and Image Understanding*, 163:78–89, 2017. 8

[15] Colin Lea, Michael D. Flynn, René Vidal, Austin Reiter, and Gregory D. Hager. Temporal convolutional networks for action segmentation and detection. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1003–1012, 2017. 1, 3, 5, 8

[16] Colin Lea, Austin Reiter, René Vidal, and Gregory D. Hager. Segmental spatiotemporal cnns for fine-grained action segmentation. In *European Conference on Computer Vision (ECCV)*, page 36–52, 2016. 8

[17] Peng Lei and Sinisa Todorovic. Temporal deformable residual networks for action segmentation in videos. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 6742–6751, 2018. 1, 3, 8

[18] Joe Yue-Hei Ng, Matthew Hausknecht, Sudheendra Vijayanarasimhan, Oriol Vinyals, Rajat Monga, and George Toderici. Beyond short snippets: Deep networks for video classification. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4694–4702, 2015. 3

[19] Hamed Pirsiavash and Deva Ramanan. Parsing videos of actions with segmental grammars. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, page 612–619, 2014. 3

[20] Alexander Richard and Juergen Gall. Temporal action detection using a statistical language model. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 3131–3140, 2016. 8

[21] Qinfeng Shi, Li Cheng, Li Wang, and Alex Smola. Human action segmentation and recognition using discriminative semi-Markov models. *International Journal of Computer Vision (IJCV)*, 93:22–32, 2011. 3

[22] Bharat Singh, Tim K. Marks, Michael Jones, Oncel Tuzel, and Ming Shao. A multi-stream bi-directional recurrent neural network for fine-grained action detection. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1961–1970, 2016. 3, 8

[23] Sebastian Stein and Stephen J. McKenna. Combining embedded accelerometers with computer vision for recognizing food preparation activities. In *Proceedings of ACM International Joint Conference on Pervasive and Ubiquitous Computing*, page 729–738, 2013. 1, 2, 5

[24] Kevin Tang, Li Fei-Fei, and Daphne Koller. Learning latent temporal structure for complex event detection. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1250–1257, 2012. 3

[25] Lingling Tao, Luca Zappella, Gregory Hager, and René Vidal. Surgical gesture segmentation and recognition. In *International Conference on Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, volume 16, pages 339–46, 09 2013. 3

[26] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, undefinedukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Proceedings of the International Conference on Neural Information Processing Systems (NIPS)*, page 6000–6010, 2017. 2, 3

[27] Oriol Vinyals, Alexander Toshev, Samy Bengio, and Dumitru Erhan. Show and tell: A neural image caption generator. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 3156–3164, 2015. 3

[28] Xiaolong Wang, Ross Girshick, Abhinav Gupta, and Kaiming He. Non-local neural networks. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7794–7803, 2018. 2, 3

[29] Zhenzhi Wang, Ziteng Gao, Limin Wang, Zhifeng Li, and Gangshan Wu. Boundary-aware cascade networks for temporal action segmentation. In *European Conference on Computer Vision (ECCV)*, 2020. 1, 2, 3, 8

[30] Serena Yeung, Olga Russakovsky, Ning Jin, Mykhaylo Andriluka, Greg Mori, and Li Fei-Fei. Every moment counts: Dense detailed labeling of actions in complex videos. *International Journal of Computer Vision (IJCV)*, 126(2):375–389, 2018. 3

[31] Yuhui Yuan and Jingdong Wang. OCNet: Object context network for scene parsing, 2019. 2, 3

[32] Hengshuang Zhao, Jiaya Jia, and Vladlen Koltun. Exploring self-attention for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 10073–10082, 2020. 2, 3